

Conceptual Data Mapping: IDTA 02004-2-0 Handover Documentation ↔ OpenUSD (Bidirectional)

```
**{octicon}`tag;1em` Document Version:** {bdg-secondary}`0.1.0`  
<br>**{octicon}`calendar;1em` Last Update:** {bdg-secondary}`2026-04-11`
```

Introduction

Overview

This document defines a **bidirectional** mapping between the **IDTA 02004-2-0 Handover Documentation** Submodel Template and OpenUSD. It covers both translation directions:

- **AAS → USD:** How AAS Submodel data is encoded in USD using the `HandoverDocumentationAPI`, `DocumentAPI`, `DocumentVersionAPI`, `DocumentIdAPI`, and `DocumentClassificationAPI` applied schemas.
- **USD → AAS:** How USD prim data is read back and reconstructed into a conformant `HandoverDocumentation` AAS Submodel instance.

The Handover Documentation submodel standardises the transfer of product-related documents — operating manuals, certificates, drawings, compliance declarations — between asset owners, operators, and service organisations. The submodel is structured around a `Documents` SML, where each `Document` SMC carries one or more identifiers from different document management systems, one or more classifications per standard (e.g. VDI 2770:2020), and one or more `DocumentVersion` SMCs containing the actual files.

This mapping is a concrete example supporting the **Identifier Separation of Concerns** proposal: the `DocumentIds` SML is a multi-system identifier store where document IDs from DMS, ERP, and regulatory systems must survive cross-system transit verbatim — they must not be interpreted as USD file paths or otherwise mangled by path-resolution semantics.

Relationship to Battery Passport and Digital Nameplate Mappings

The IDTA 02035-1 Digital Battery Passport references this submodel for compliance documents: the `EUDeclarationOfConformity` and `ResultsOfTestReportsProvingCompliance` fields in the battery passport nameplate store **document identifier strings** that reference `DocumentIds__NN__/DocumentIdentifier` entries in this submodel. When both submodels are co-located in the same USD stage, the cross-reference can be made explicit. See [Appendix D](#) for details.

```
**Dependency on the Source Identifier proposal.** The `sourceId:doc:*` mechanism used in  
this document is defined in the [Identifier Separation of Concerns proposal](./README.md)  
and is not yet part of the OpenUSD standard.
```

References

AAS / IDTA Reference

Version	Reference Documents
2.0 (2023)	IDTA 02004-2-0: Submodel Template Specification — Handover Documentation
—	VDI 2770 Part 1 (2020): Minimum requirements for digital manufacturer information for the lifecycle of machines
—	IEC 61355-1: Classification and designation of documents for plants, systems and equipment
—	IDTA-01001-3-1-2: Specification of the Asset Administration Shell Part 1: Metamodel (V3.1.2)

OpenUSD Reference

Version	Reference Documents
24.08	OpenUSD C++ and Schema Documentation , OpenUSD Github Repository .
— (proposed)	Identifier Separation of Concerns — defines the <code>sourceId:*</code> mechanism used in this document

General Assumptions and Constraints

- **Two-way mapping** (AAS ↔ USD). Round-trip fidelity is a goal but not fully achievable; lossy fields are explicitly documented.
- The `HandoverDocumentation` submodel is encoded using five applied API schemas. Schema definitions are in [HandoverDocumentation \(Submodel\)](#).
- `HandoverDocumentationAPI` may be applied to a product prim alongside `DppNameplateAPI` (co-located with product identity metadata) or to a standalone `HandoverDocumentation` scope prim in a separate documentation layer. Both patterns are valid; the child prim hierarchy is the same in both cases.
- `Documents` **SML** maps to a `Documents` child scope prim directly under the prim that has `HandoverDocumentationAPI` applied. This mirrors the `Markings/Marking_NN` pattern from the nameplate mappings.
- **Document SMC idShort** in AAS instances may be user-defined (e.g., `Datasheet`, `Manual`). In USD, the prim is named `Document_NN` (zero-padded index preserving SML order); the original AAS idShort is preserved in `customData["aas:idShort"]`.
- **SML ordering** throughout is preserved by zero-padded numeric suffixes (`Document_NN`, `DocumentVersion_NN`, `DocumentId_NN`, `DocumentClassification_NN`).
- **MLP fields** (`Title`, `Subtitle`, `Description`, `Keywords`, `ClassName`): USD stores only the primary language string value. Language variants go into `customData["doc:<field>i18n"]` dictionaries.
- `DocumentedEntities` SML: AAS references to documented assets. In USD, intra-stage references →

`rel doc:documentedEntities`; AAS external entity IDs → `sourceId:doc:documentedEntityIds` (string[]).

- **Document relationships** (`RefersToEntities`, `BasedOnReferences`, `TranslationOfEntities`): USD `rel` attributes for intra-stage links; `sourceId:doc:*` string attributes for cross-stage or cross-system links. Both may be authored simultaneously.
- **MIME types**: AAS `File.contentType` has no direct equivalent in the USD `asset` type. Store MIME types in parallel `customData` entries when round-trip fidelity is required.
- **Entities SML** (top-level, optional): The VDI 2770 concept of documenting the product entity hierarchy in AAS `Entity` elements. In USD, product structure is already represented as prim hierarchies; this SML has no direct USD mapping and is out of scope for this document.
- `semanticIds` are preserved in `customData` for provenance and survive round-trip only if USD tooling does not strip `customData`.
- **AAS structural context** (Asset, AssetAdministrationShell, Submodel containment) is not encoded in USD. USD → AAS produces a Submodel instance only.

Definitions, Acronyms, Abbreviations

Term	Description
AAS	Asset Administration Shell
SMT	Submodel Template
SMC	SubmodelElementCollection
SML	SubmodelElementList
MLP	MultiLanguageProperty
DMS	Document Management System
<code>HandoverDocumentationAPI</code>	Applied USD API schema signalling IDTA 02004-2-0 handover documentation content
<code>DocumentAPI</code>	Applied USD API schema for a single <code>Document</code> SML entry
<code>DocumentVersionAPI</code>	Applied USD API schema for a single <code>DocumentVersion</code> SML entry
<code>DocumentIdAPI</code>	Applied USD API schema for a single document identifier entry
<code>DocumentClassificationAPI</code>	Applied USD API schema for a single document classification entry
<code>doc:*</code>	Attribute namespace prefix shared by <code>DocumentVersionAPI</code> , <code>DocumentIdAPI</code> (<code>docId:*</code>), and <code>DocumentClassificationAPI</code> (<code>docClass:*</code>)
<code>sourceId:doc:*</code>	Source identifier attributes for document-scoped cross-system identifiers

Concepts

High-Level Concept Mapping

AAS Element	OpenUSD	Round-trip	Notes
HandoverDocumentation (Submodel)	Prim with <code>HandoverDocumentationAPI</code> + <code>Documents</code> child scope	Partial	AAS envelope not round-tripped
Documents (SML)	<code>Documents</code> scope prim, <code>Document_NN</code> children with <code>DocumentAPI</code>	Partial	AAS Document idShort preserved in <code>customData</code>
DocumentIds (SML)	<code>DocumentIds/DocumentId_NN</code> children with <code>DocumentIdAPI</code>	Lossless	Primary ID also in <code>sourceId:doc:documentId</code>
DocumentClassifications (SML)	<code>DocumentClassifications/DocumentClassification_NN</code> children with <code>DocumentClassificationAPI</code>	Partial	ClassName MLP lossy
DocumentedEntities (SML)	<code>rel doc:documentedEntities</code> + <code>sourceId:doc:documentedEntityIds</code>	Partial	AAS refs via source id
DocumentVersions (SML)	<code>DocumentVersions/DocumentVersion_NN</code> children with <code>DocumentVersionAPI</code>	Partial	—
Language (SML)	<code>doc:languages</code> (string[])	Lossless	BCP 47 codes
Version	<code>doc:version</code> (string)	Lossless	e.g. "v1.2"
Title	<code>doc:title</code> (string)	Lossy	Primary language only
Subtitle	<code>doc:subtitle</code> (string)	Lossy	Optional; not authored if absent
Description	<code>doc:description</code> (string)	Lossy	Primary language only; mandatory
KeyWords	<code>doc:keywords</code> (string)	Lossy	Primary language only; optional
StatusValue	<code>doc:status</code> (token)	Lossless	"Released" / "InReview" / "Withdrawn"
StatusSetDate	<code>doc:statusSetDate</code> (string)	Lossless	ISO 8601 YYYY-MM-DD
OrganizationShortName	<code>doc:organizationShortName</code> (string)	Lossless	Mandatory
OrganizationOfficialName	<code>doc:organizationOfficialName</code> (string)	Lossless	Mandatory
DigitalFiles (SML[File])	<code>doc:digitalFiles</code> (asset[])	Partial	MIME type lossy
PreviewFile (File)	<code>doc:previewFile</code> (asset)	Partial	MIME type lossy; optional
RefersToEntities (SML)	<code>rel doc:refersToEntities</code> / <code>sourceId:doc:refersToEntityIds</code>	Partial	Cross-stage: source id
BasedOnReferences (SML)	<code>rel doc:basedOnReferences</code> / <code>sourceId:doc:basedOnReferenceIds</code>	Partial	Cross-stage: source id
TranslationOfEntities (SML)	<code>rel doc:translationOfEntities</code> / <code>sourceId:doc:translationOfEntityIds</code>	Partial	Cross-stage: source id
Entities (SML)	—	Dropped	VDI 2770 entity hierarchy; USD prim hierarchy replaces this concept
—	Composition arcs	Dropped	No AAS equivalent
—	Time-varying attributes	Dropped	No AAS equivalent
—	Variant sets	Dropped	No AAS equivalent

HandoverDocumentation (Submodel)

The `HandoverDocumentation` AAS Submodel (semanticId: 0173-1#01-AHF578#003) maps to a USD prim with `HandoverDocumentationAPI` applied, with a `Documents` child scope prim containing the `Document_NN` entries.

Schema Definitions (illustrative)

```
# — Handover Documentation submodel marker —————
class "HandoverDocumentationAPI" (
  inherits = </APISchemaBase>
  doc = "Signals IDTA 02004-2-0 Handover Documentation content. Applied to a product prim
  alongside DppNameplateAPI / IndustrialEquipmentAPI, or to a standalone HandoverDocumentation
  scope prim in a documentation-only layer. Document entries live in a Documents/ child scope
  prim containing Document_NN children with DocumentAPI applied."
  customData = {
    token apiSchemaType = "singleApply"
  }
) {
  # No direct attributes.
  # This schema is a discoverer: tooling can find all prims carrying
  # handover documentation by checking for HandoverDocumentationAPI.
}

# — Per-document entry —————
class "DocumentAPI" (
  inherits = </APISchemaBase>
  doc = "Applied to Document_NN child scope prims under the Documents/ scope. Each
  instance corresponds to one Document SMC entry in the IDTA 02004-2-0 Documents SML. Content
  lives in DocumentIds/, DocumentClassifications/, and DocumentVersions/ child scope prims.
  The original AAS idShort (e.g. 'Datasheet') is preserved in customData['aas:idShort']."
  customData = {
    token apiSchemaType = "singleApply"
  }
) {
  rel doc:documentedEntities (
    doc = "USD relationships to prims this document covers (intra-stage). AAS:
    DocumentedEntities (SML[ReferenceElement]). For cross-stage or AAS external refs use
    sourceId:doc:documentedEntityIds (string[])."
  )
}

# — Per-document-version entry —————
class "DocumentVersionAPI" (
  inherits = </APISchemaBase>
  doc = "Applied to DocumentVersion_NN child scope prims under
  Document_NN/DocumentVersions/. Encodes a single DocumentVersion SMC from the
  DocumentVersions SML."
  customData = {
    token apiSchemaType = "singleApply"
  }
) {
  string[] doc:languages (
    doc = "BCP 47 language codes for this document version, e.g. ['en', 'de']. (AAS:
    Language, SML of xs:string Property, mandatory)"
  )
  string doc:version (
```

```

        doc = "Version designation string, e.g. 'v1.2'. (AAS: Version, xs:string,
mandatory)"
    )
    string doc:title (
        doc = "Document title in the primary language. Language variants in
customData['doc:title:i18n']. (AAS: Title, MLP, mandatory)"
    )
    string doc:subtitle (
        doc = "Document subtitle in the primary language. Not authored if absent. (AAS:
Subtitle, MLP, optional)"
    )
    string doc:description (
        doc = "Short description or abstract in the primary language. Language variants in
customData['doc:description:i18n']. (AAS: Description, MLP, mandatory)"
    )
    string doc:keywords (
        doc = "Keywords string in the primary language (space- or comma-separated). Not
authored if absent. (AAS: Keywords, MLP, optional)"
    )
    token doc:status (
        allowedTokens = ["Released", "InReview", "Withdrawn"]
        doc = "Lifecycle status of this document version. (AAS: StatusValue, xs:string,
mandatory)"
    )
    string doc:statusSetDate (
        doc = "Date the current status was set, ISO 8601 YYYY-MM-DD. (AAS: StatusSetDate,
xs:date, mandatory)"
    )
    string doc:organizationShortName (
        doc = "Short name of the issuing organisation. (AAS: OrganizationShortName,
xs:string, mandatory)"
    )
    string doc:organizationOfficialName (
        doc = "Official legal name of the issuing organisation. (AAS:
OrganizationOfficialName, xs:string, mandatory)"
    )
    asset[] doc:digitalFiles (
        doc = "File assets for this document version. Multiple entries for multi-format
delivery (e.g. PDF + XML). (AAS: DigitalFiles, SML of File, mandatory)"
    )
    asset doc:previewFile (
        doc = "Preview or thumbnail image of the document. Not authored if absent. (AAS:
PreviewFile, File, optional)"
    )
    rel doc:refersToEntities (
        doc = "Related document version prims (intra-stage). AAS: RefersToEntities
(SML[ReferenceElement]). For cross-stage use sourceId:doc:refersToEntityIds (string[])."
    )
    rel doc:basedOnReferences (
        doc = "Document versions this version is derived from (intra-stage). AAS:
BasedOnReferences (SML[ReferenceElement]). For cross-stage use
sourceId:doc:basedOnReferenceIds (string[])."
    )

```

```

    rel doc:translationOfEntities (
        doc = "Source-language document versions this version translates (intra-stage). AAS:
TranslationOfEntities (SML[ReferenceElement]). For cross-stage use
sourceId:doc:translationOfEntityIds (string[])."
```

)

```

}

# — Per-document-identifier entry —————
class "DocumentIdAPI" (
    inherits = </APISchemaBase>
    doc = "Applied to DocumentId_NN child scope prims under Document_NN/DocumentIds/. Each
instance corresponds to one SMC entry in the DocumentIds SML. The entry where
docId:isPrimary = true is the canonical cross-system document identifier."
    customData = {
        token apiSchemaType = "singleApply"
    }
) {
    string docId:documentDomainId (
        doc = "The domain or system that issued this identifier, e.g.
'https://www.example.com/aas/'. (AAS: DocumentDomainId, xs:string, mandatory)"
    )
    string docId:documentIdentifier (
        doc = "The document identifier value within the domain. Must survive cross-system
transit verbatim. (AAS: DocumentIdentifier, xs:string, mandatory)"
    )
    bool docId:isPrimary (
        doc = "True if this is the primary/canonical identifier for the document. Only one
entry per Document should have isPrimary = true. (AAS: DocumentIsPrimary, xs:boolean,
optional)"
    )
}

# — Per-classification entry —————
class "DocumentClassificationAPI" (
    inherits = </APISchemaBase>
    doc = "Applied to DocumentClassification_NN child scope prims under
Document_NN/DocumentClassifications/. Each instance corresponds to one SMC entry in the
DocumentClassifications SML. A document may carry classifications in multiple systems."
    customData = {
        token apiSchemaType = "singleApply"
    }
) {
    string docClass:classificationSystem (
        doc = "Classification system name, e.g. 'VDI2770:2020', 'IEC61355-1'. (AAS:
ClassificationSystem, xs:string, mandatory) See Appendix B for common VDI 2770:2020 codes."
    )
    string docClass:classId (
        doc = "Class code within the system, e.g. '03-02'. (AAS: ClassId, xs:string,
mandatory)"
    )
    string docClass:className (
        doc = "Human-readable class name, primary language. Language variants in
customData['docClass:className:i18n']. (AAS: ClassName, MLP, mandatory)"
    )
}

```

```
    )  
  }
```

Source Identifier Entries

Two source identifier entries are authored on each `Document_NN` prim:

Source Identifier	USD Type	Description
<code>sourceId:doc:documentId</code>	<code>string</code>	The <code>DocumentIdentifier</code> from the <code>DocumentIdAPI</code> entry where <code>docId:isPrimary = true</code> . The canonical cross-system document reference key.
<code>sourceId:doc:documentDomainId</code>	<code>string</code>	The <code>DocumentDomainId</code> of the primary identifier. Context for interpreting <code>sourceId:doc:documentId</code> .

These are a **convenience projection** of the primary identifier. Tooling that does not understand `DocumentIdAPI` can discover the canonical document ID without traversing the `DocumentIds/` child prim hierarchy. The full multi-system identifier set in the child prims is authoritative.

Prim Hierarchy

```
Product (Xform, DppNameplateAPI + IndustrialEquipmentAPI + HandoverDocumentationAPI)  
└─ Documents (Scope)                                     ← AAS: Documents SML  
    └─ Document_00 (Scope, DocumentAPI)                   ← AAS: Document SMC (e.g. idShort  
        "Datasheet")  
            └─ DocumentIds (Scope)                         ← AAS: DocumentIds SML  
                └─ DocumentId_00 (Scope, DocumentIdAPI)   # isPrimary = true  
                └─ DocumentId_01 (Scope, DocumentIdAPI)  
            └─ DocumentClassifications (Scope)             ← AAS: DocumentClassifications SML  
                └─ DocumentClassification_00 (Scope, DocumentClassificationAPI)  
            └─ DocumentVersions (Scope)                     ← AAS: DocumentVersions SML  
                └─ DocumentVersion_00 (Scope, DocumentVersionAPI)  
    └─ Document_01 (Scope, DocumentAPI)  
    ...
```

Standalone documentation prim (documentation-only USD file, separate from product geometry):


```
def Scope "HandoverDocumentation" (
  prepend apiSchemas = ["HandoverDocumentationAPI"]
  customData = {
    string "aas:submodelSemanticId" = "0173-1#01-AHF578#003"
  }
) {
  def Scope "Documents" {
    def Scope "Document_00" (prepend apiSchemas = ["DocumentAPI"]) {
      ...
    }
  }
}
```

AAS → USD

- 1. Apply HandoverDocumentationAPI to the root prim (product or standalone).
- 2. Preserve submodelSemanticId (0173-1#01-AHF578#003) in customData["aas:submodelSemanticId"].
- 3. Create a Documents child scope prim.
- 4. For each Document SMC in the AAS Documents SML: create child prim Document_NN (zero-padded index preserving SML order) with DocumentAPI applied. Preserve the AAS idShort in customData["aas:idShort"].
- 5. Write sourceId:doc:documentId and sourceId:doc:documentDomainId on each Document_NN prim from the primary identifier (the entry where DocumentIsPrimary = true , or the sole entry if only one is present).

DocumentIds

The DocumentIds SML contains one or more SMCs, each identifying the document in a different domain or system. Where a document has multiple IDs, the one with DocumentIsPrimary = true is the canonical identifier for cross-system referencing.

This is the central identifier use case for the Identifier Separation of Concerns proposal: document IDs from DMS, ERP, and regulatory systems are opaque strings that must survive cross-system transit without path-resolution side effects.

AAS → USD

Each DocumentId SMC → child scope prim DocumentId_NN under Document_NN/DocumentIds/ , with DocumentIdAPI applied (zero-padded index, preserving SML order). Write source id entries on the Document_NN prim from the primary identifier.

Properties (per DocumentId entry)

AAS idShort	USD Attribute	AAS Type	USD Type	Notes
DocumentDomainId	docId:documentDomainId	xs:string	string	Mandatory

AAS idShort	USD Attribute	AAS Type	USD Type	Notes
DocumentIdentifier	docId:documentIdentifier	xs:string	string	Mandatory
DocumentIsPrimary	docId:isPrimary	xs:boolean	bool	Optional (ZeroToOne); omit if absent

Usage Example

```
def Scope "Document_00" (  
  prepend apiSchemas = ["DocumentAPI"]  
  customData = { string "aas:idShort" = "Datasheet" }  
) {  
  custom string sourceId:doc:documentId      = "123-ABC-456"  
  custom string sourceId:doc:documentDomainId = "https://www.aasexample.com/aas/"  
  
  def Scope "DocumentIds" {  
    def Scope "DocumentId_00" (prepend apiSchemas = ["DocumentIdAPI"]) {  
      string docId:documentDomainId  = "https://www.aasexample.com/aas/"  
      string docId:documentIdentifier = "123-ABC-456"  
      bool   docId:isPrimary          = true  
    }  
    def Scope "DocumentId_01" (prepend apiSchemas = ["DocumentIdAPI"]) {  
      string docId:documentDomainId  = "VDI2770"  
      string docId:documentIdentifier = "MUSTER-AG_DS_CM5-3_2024_EN"  
      # isPrimary omitted (absent in AAS)  
    }  
  }  
  ...  
}
```

USD → AAS

1. Collect `DocumentId_NN` children under `DocumentIds/`; sort by name to recover SML order.
2. For each: construct a DocumentId SMC with `DocumentDomainId`, `DocumentIdentifier`. Add `DocumentIsPrimary` only when `docId:isPrimary` is authored.
3. `sourceId:doc:documentId` / `sourceId:doc:documentDomainId` are convenience projections; child prims are authoritative.

Round-trip

Field	Fidelity
DocumentDomainId	Lossless
DocumentIdentifier	Lossless
DocumentIsPrimary	Lossless (when authored)
SML ordering	Lossless (requires <code>DocumentId_NN</code> naming convention)

DocumentClassification

The `DocumentClassifications` SML contains one or more classification SMCs. A document may be classified in multiple systems simultaneously (e.g., both VDI 2770:2020 and IEC 61355-1).

AAS → USD

Each DocumentClassification SMC → child scope prim `DocumentClassification_NN` under `Document_NN/DocumentClassifications/`, with `DocumentClassificationAPI` applied.

Properties (per DocumentClassification entry)

AAS idShort	USD Attribute	AAS Type	USD Type
ClassificationSystem	<code>docClass:classificationSystem</code>	<code>xs:string</code>	<code>string</code>
ClassId	<code>docClass:classId</code>	<code>xs:string</code>	<code>string</code>
ClassName	<code>docClass:className</code>	MLP	<code>string</code> (primary language)

Language variants of `ClassName` → `customData["docClass:className:i18n"]` on the prim.

Usage Example

```
def Scope "DocumentClassifications" {
  def Scope "DocumentClassification_00" (prepend apiSchemas =
["DocumentClassificationAPI"]) {
    string docClass:classificationSystem = "VDI2770:2020"
    string docClass:classId              = "06-01"
    string docClass:className             = "Certificates and declarations of conformity"
    customData = {
      dictionary "docClass:className:i18n" = {
        string "de" = "Zertifikate und Konformitätserklärungen"
        string "en" = "Certificates and declarations of conformity"
      }
    }
  }
}
```

See [Appendix B](#) for VDI 2770:2020 class codes.

USD → AAS

1. Collect `DocumentClassification_NN` children under `DocumentClassifications/`; sort by name.
2. For each: construct a DocumentClassification SMC with `ClassificationSystem`, `ClassId`, and `ClassName` MLP (primary + i18n dict variants).

Round-trip

Field	Fidelity
ClassificationSystem	Lossless
ClassId	Lossless
ClassName (primary language)	Lossless
ClassName (language variants)	Lossy — conditional on <code>customData</code> not stripped

DocumentedEntities

An optional SML of `ReferenceElement` entries identifying which assets or asset components this document covers (VDI 2770: `DocumentedEntity`). Allows a single document to be associated with one or more product instances or sub-components. In AAS these are `ModelReference` or `ExternalReference` elements.

Two encoding strategies apply in USD:

Strategy A — intra-stage (preferred when the product prim is in the same stage):

```
re1 doc:documentedEntities = </Products/Pump_MusterAG_CM5_3>
```

Strategy B — cross-stage / AAS external reference strings:

```
custom string[] sourceId:doc:documentedEntityIds = [
  "https://products.muster-ag.de/cm5-3/SN-20240001"
]
```

Both may be authored simultaneously for provenance.

AAS → USD

If the referenced asset's ID matches a USD prim in the current stage, write Strategy A. Always write Strategy B with the verbatim AAS reference strings when present.

USD → AAS

If `sourceId:doc:documentedEntityIds` is present: reconstruct as `ExternalReference` SML entries.
If only `re1 doc:documentedEntities` is present: resolve relationship targets and extract their primary asset identifier (e.g., `sourceId:dpp:uriOfTheProduct`).

Round-trip

	Fidelity
AAS <code>ExternalReference</code> strings	Lossless via <code>sourceId:doc:documentedEntityIds</code>
AAS <code>ModelReference</code> structure	Partial — leaf identifier string only

DocumentVersion

Each DocumentVersion SMC maps to a child scope prim `DocumentVersion_NN` under `Document_NN/DocumentVersions/`, with `DocumentVersionAPI` applied. Zero-padded index preserves SML ordering.

Language

The `Language` SML contains one or more language code string properties indicating the language(s) of this document version. Codes follow BCP 47.

Properties

AAS idShort	USD Attribute	AAS Type	USD Type
Language (SML)	<code>doc:languages</code>	SML of <code>xs:string</code>	<code>string[]</code>

Round-trip: Lossless

Version

The document version designation string (e.g., `v1.2`, `Rev. A`, `1.0`). Mandatory.

Properties

AAS idShort	USD Attribute	AAS Type	USD Type
Version	<code>doc:version</code>	<code>xs:string</code>	<code>string</code>

Round-trip: Lossless

Title / Subtitle / Description / KeyWords

All four are MLP fields. USD stores the primary language value; language variants go into `customData`.

AAS idShort	USD Attribute	USD Type	Cardinality
<code>Title</code>	<code>doc:title</code>	<code>string</code>	mandatory
<code>Subtitle</code>	<code>doc:subtitle</code>	<code>string</code>	optional (0..1); not authored if absent
<code>Description</code>	<code>doc:description</code>	<code>string</code>	mandatory
<code>KeyWords</code>	<code>doc:keywords</code>	<code>string</code>	optional (0..1); not authored if absent

`keywords` contains free-text keywords in MLP form — mapped to a plain `string` (primary language).
Language variants: `customData["doc:keywords:i18n"]`.

Round-trip: Lossy — primary language lossless; language variants conditional on `customData`

StatusValue / StatusSetDate

`StatusValue` is a controlled vocabulary string for the lifecycle state of this document version. `StatusSetDate` is the ISO 8601 date the status was last changed.

AAS → USD

Map `StatusValue` string to the corresponding `doc:status` token (1:1). Write `StatusSetDate` as ISO 8601 string.

Properties

AAS idShort	USD Attribute	AAS Type	USD Type
StatusValue	<code>doc:status</code>	<code>xs:string</code> enum	<code>token</code> (<code>allowedTokens</code> = ["Released", "InReview", "Withdrawn"])
StatusSetDate	<code>doc:statusSetDate</code>	<code>xs:date</code>	<code>string</code> (ISO 8601 YYYY-MM-DD)

See [Appendix A](#) for the token mapping table.

USD → AAS

Read `doc:status` token → write `StatusValue` Property string. Read `doc:statusSetDate` → write `StatusSetDate` Property with `valueType` = `xs:date`.

Round-trip: Lossless

OrganizationShortName / OrganizationOfficialName

Both are mandatory plain `xs:string` (not MLP). `OrganizationShortName` is the abbreviated name used for display; `OrganizationOfficialName` is the full legal name.

Properties

AAS idShort	USD Attribute	AAS Type	USD Type	Notes
OrganizationShortName	<code>doc:organizationShortName</code>	<code>xs:string</code>	<code>string</code>	Mandatory
OrganizationOfficialName	<code>doc:organizationOfficialName</code>	<code>xs:string</code>	<code>string</code>	Mandatory

Round-trip: Lossless

DigitalFiles

The `DigitalFiles` SML contains one or more AAS `File` elements — the actual document files for this version, potentially in multiple formats (e.g., PDF and XML). Each `File` carries a path and a MIME type (`contentType`).

AAS → USD

Write all file paths to `doc:digitalFiles` (asset[]). Multiple formats → separate array elements. The AAS `File.contentType` has no direct equivalent in the USD `asset` type; when MIME type round-trip is required, store per-file MIME type strings in `customData["doc:digitalFilesMimeTypes"]` as a string[] parallel to `doc:digitalFiles`.

Properties

AAS	USD Attribute	AAS Type	USD Type
DigitalFiles SML entries (paths)	<code>doc:digitalFiles</code>	SML of <code>File</code>	<code>asset[]</code>
DigitalFiles SML entries (MIME)	<code>customData["doc:digitalFilesMimeTypes"]</code>	<code>xs:string</code> (MIME)	<code>string[]</code>

USD → AAS

Read `doc:digitalFiles` (asset[]). For each, construct a DigitalFile entry. Set `contentType` from `customData["doc:digitalFilesMimeTypes"]` (parallel index) if present; otherwise `"application/octet-stream"`.

Round-trip

	Fidelity
File paths	Lossless
File array ordering	Lossless
MIME types	Lossy — lossless only when <code>customData["doc:digitalFilesMimeTypes"]</code> authored

PreviewFile

An optional preview image of the document. AAS type `File` (single, with MIME type).

Properties

AAS idShort	USD Attribute	AAS Type	USD Type
PreviewFile path	doc:previewFile	File	asset
PreviewFile MIME	customData["doc:previewFileMimeType"]	xs:string	string

Round-trip: Lossless for path; MIME type lossy unless
`customData["doc:previewFileMimeType"]` **authored**

Document Relationships

Three optional SML fields in DocumentVersion encode relationships to other document versions:

AAS idShort	Meaning	Cardinality
RefersToEntities	Related documents	0..*
BasedOnReferences	This version is derived from / approved based on other versions	0..*
TranslationOfEntities	This version is a translation of source-language versions	0..*

All three contain `ReferenceElement` entries pointing to other DocumentVersion SMCs.

Strategy A — intra-stage (preferred):

```
def Scope "DocumentVersion_00" (prepend apiSchemas = ["DocumentVersionAPI"]) {
  rel doc:refersToEntities      =
  [</HandoverDocumentation/Documents/Document_01/DocumentVersions/DocumentVersion_00>]
  rel doc:basedOnReferences    =
  </HandoverDocumentation/Documents/Document_00/DocumentVersions/DocumentVersion_00>
  rel doc:translationOfEntities =
  </HandoverDocumentation/Documents/Document_00/DocumentVersions/DocumentVersion_00>
  ...
}
```

Strategy B — cross-stage / cross-system:

```
custom string[] sourceId:doc:refersToEntityIds      = ["DOC-2023-BASE-001"]
custom string[] sourceId:doc:basedOnReferenceIds    = ["DOC-2022-ORIG-EN-001"]
custom string[] sourceId:doc:translationOfEntityIds = ["DOC-2022-ORIG-EN-001"]
```

Both strategies may be co-authored (Strategy A for USD links, Strategy B for provenance).

AAS → USD

For each ReferenceElement: if the target DocumentVersion prim is in the current stage, write Strategy A + Strategy B source ids. If cross-stage, write Strategy B only.

USD → AAS

Strategy A: resolve target prims, read `sourceId:doc:documentId` as external ID, construct ReferenceElement.
Strategy B: use string values directly.

Round-trip

	Fidelity
Referenced document identifier values	Lossless (Strategy B)
Intra-stage references	Lossless (requires naming convention + Strategy A)

Full Usage Example

```
def xform "Pump_MusterAG_CM5_3" (
  kind = "component"
  prepend apiSchemas = ["DppNameplateAPI", "IndustrialEquipmentAPI",
    "HandoverDocumentationAPI"]
  customData = {
    string "aas:submodelSemanticId:handoverDoc" = "0173-1#01-AHF578#003"
  }
) {
  # DppNameplateAPI + IndustrialEquipmentAPI fields omitted for brevity ...

  def Scope "Documents" {

    # — EU Declaration of Conformity —————
    def Scope "Document_00" (
      prepend apiSchemas = ["DocumentAPI"]
      customData = { string "aas:idShort" = "EUDeclaration" }
    ) {
      custom string sourceId:doc:documentId      = "DOC-2024-CE-001"
      custom string sourceId:doc:documentDomainId = "SAP-DMS"

      rel doc:documentedEntities = </Pump_MusterAG_CM5_3>
      custom string[] sourceId:doc:documentedEntityIds = [
        "https://products.muster-ag.de/cm5-3/SN-20240001"
      ]

      def Scope "DocumentIds" {
        def Scope "DocumentId_00" (prepend apiSchemas = ["DocumentIdAPI"]) {
          string docId:documentDomainId      = "SAP-DMS"
          string docId:documentIdentifier = "DOC-2024-CE-001"
          bool docId:isPrimary              = true
        }
      }
    }
  }
}
```

```

def Scope "DocumentId_01" (prepend apiSchemas = ["DocumentIdAPI"]) {
    string docId:documentDomainId = "VDI2770"
    string docId:documentIdentifier = "MUSTER-AG_CE_Declaration_2024_EN_DE"
}

def Scope "DocumentClassifications" {
    def Scope "DocumentClassification_00" (prepend apiSchemas =
["DocumentClassificationAPI"]) {
        string docClass:classificationSystem = "VDI2770:2020"
        string docClass:classId = "06-01"
        string docClass:className = "Certificates and declarations of
conformity"
    }
}

def Scope "DocumentVersions" {
    def Scope "DocumentVersion_00" (
        prepend apiSchemas = ["DocumentVersionAPI"]
        customData = {
            dictionary "doc:title:i18n" = {
                string "de" = "EU-Konformitätserklärung"
                string "en" = "EU Declaration of Conformity"
            }
            dictionary "doc:description:i18n" = {
                string "de" = "EU-Konformitätserklärung für die CM5-3-
Kreiselpumpe"
                string "en" = "EU Declaration of Conformity for the CM5-3
centrifugal pump"
            }
            string[] "doc:digitalFilesMimeTypes" = ["application/pdf",
"application/pdf"]
        }
    ) {
        string[] doc:languages = ["en", "de"]
        string doc:version = "1.0"
        string doc:title = "EU Declaration of Conformity"
        string doc:description = "EU Declaration of Conformity for
the CM5-3 centrifugal pump"
        token doc:status = "Released"
        string doc:statusSetDate = "2024-01-15"
        string doc:organizationShortName = "Muster AG"
        string doc:organizationOfficialName = "Muster Aktiengesellschaft"
        asset[] doc:digitalFiles = [
            @./docs/eu_declaration_en.pdf@,
            @./docs/eu_declaration_de.pdf@
        ]
        asset doc:previewFile = @./docs/eu_declaration_preview.jpg@
    }
}

```

```

def Scope "Document_01" (
  prepend apiSchemas = ["DocumentAPI"]
  customData = { string "aas:idShort" = "OperatingManual" }
) {
  custom string sourceId:doc:documentId      = "MAN-CM5-3-001"
  custom string sourceId:doc:documentDomainId = "SAP-DMS"

  def Scope "DocumentIds" {
    def Scope "DocumentId_00" (prepend apiSchemas = ["DocumentIdAPI"]) {
      string docId:documentDomainId = "SAP-DMS"
      string docId:documentIdentifier = "MAN-CM5-3-001"
      bool  docId:isPrimary          = true
    }
  }

  def Scope "DocumentClassifications" {
    def Scope "DocumentClassification_00" (prepend apiSchemas =
["DocumentClassificationAPI"]) {
      string docClass:classificationSystem = "VDI2770:2020"
      string docClass:classId              = "03-02"
      string docClass:className            = "Operating manual"
    }
  }

  def Scope "DocumentVersions" {
    # English version
    def Scope "DocumentVersion_00" (prepend apiSchemas = ["DocumentVersionAPI"])
{
      string[] doc:languages          = ["en"]
      string   doc:version             = "v1.2"
      string   doc:title               = "Operating Manual – CM5-3
Centrifugal Pump"
      string   doc:description         = "Full operating, installation, and
maintenance instructions for the CM5-3 series."
      string   doc:keywords            = "pump centrifugal CM5-3 installation
maintenance"
      token    doc:status              = "Released"
      string   doc:statusSetDate       = "2024-03-01"
      string   doc:organizationShortName = "Muster AG"
      string   doc:organizationOfficialName = "Muster Aktiengesellschaft"
      asset[]  doc:digitalFiles        =
["@./docs/operating_manual_cm5_3_en.pdf@"]
      asset    doc:previewFile         = "@./docs/manual_cover_preview.png@"
    }
    # German translation
    def Scope "DocumentVersion_01" (
      prepend apiSchemas = ["DocumentVersionAPI"]
      customData = {
        dictionary "doc:title:i18n" = {
          string "de" = "Betriebsanleitung – CM5-3 Kreiselpumpe"
          string "en" = "Operating Manual – CM5-3 Centrifugal Pump"
        }
      }
    )
  }
}

```

```

    ) {
        string[] doc:languages          = ["de"]
        string   doc:version            = "v1.2"
        string   doc:title              = "Betriebsanleitung – CM5-3
Kreiselpumpe"

        string   doc:description        = "vollständige Betriebs-,
Installations- und Wartungsanleitung für die CM5-3-Baureihe."
        token    doc:status             = "Released"
        string   doc:statusSetDate      = "2024-03-01"
        string   doc:organizationShortName = "Muster AG"
        string   doc:organizationOfficialName = "Muster Aktiengesellschaft"
        asset[]  doc:digitalFiles       =

["@./docs/betriebsanleitung_cm5_3_de.pdf@]

        # This German version translates the English original
        rel doc:translationOfEntities =
</Pump_MusterAG_CM5_3/Documents/Document_01/DocumentVersions/DocumentVersion_00>
        custom string[] sourceId:doc:translationOfEntityIds = ["MAN-CM5-3-001"]
    }
}
}
}
}
}
}

```

USD → AAS Reconstruction

Prerequisites

- The prim has `HandoverDocumentationAPI` applied and a `Documents` child scope prim is present; or a standalone `HandoverDocumentation` scope prim is the target.
- AAS structural context (AssetAdministrationShell, Asset, Submodel envelope) must be provided by the receiving system.

Reconstruction Steps

1. **Locate the Documents scope:** Find the child scope prim named `Documents` under the prim with `HandoverDocumentationAPI`.
2. **Submodel metadata:** Set `idShort` = "HandoverDocumentation". Set `semanticId` from `customData["aas:submodelSemanticId"]` if present; otherwise use `0173-1#01-AHF578#003`.
3. **Collect documents:** Find all `DocumentAPI` children under `Documents/`; sort by prim name (`Document_NN`). Each becomes a Document SMC. Set `idShort` from `customData["aas:idShort"]` if present; otherwise use prim name.
4. **For each Document prim:**
 - **DocumentIds SML:** Collect `DocumentId_NN` children under `DocumentIds/` (sorted). Reconstruct SMCs with `DocumentDomainId` = `docId:documentDomainId`, `DocumentIdentifier` = `docId:documentIdentifier`. Add `DocumentIsPrimary` only when `docId:isPrimary` is authored.

- **DocumentClassifications SML:** Collect `DocumentClassification_NN` children (sorted). Reconstruct SMCs with `ClassificationSystem`, `ClassId`, `ClassName` (MLP from primary string + i18n dict).
- **DocumentedEntities SML:** If `sourceId:doc:documentedEntityIds` is present, write one `ExternalReference` per entry. Else resolve `rel doc:documentedEntities` targets and extract their primary asset identifiers.
- **DocumentVersions SML:** Collect `DocumentVersion_NN` children (sorted).

5. For each `DocumentVersion` prim:

- `doc:languages` → `Language` SML (one Property per code).
- `doc:version` → `Version` Property.
- `doc:title`, `doc:subtitle`, `doc:description`, `doc:keywords` → MLP Properties. Reconstruct language variants from `customData["doc:<field>:i18n"]`. Omit optional fields when not authored.
- `doc:status` token → `StatusValue` string.
- `doc:statusSetDate` → `StatusSetDate` with `valueType = xs:date`.
- `doc:organizationShortName`, `doc:organizationOfficialName` → Properties.
- `doc:digitalFiles` (asset[]) → `DigitalFiles` SML. Set `contentType` from `customData["doc:digitalFilesMimeTypes"]` (parallel index) or `"application/octet-stream"`.
- `doc:previewFile` → `PreviewFile` if authored.
- **Relationships:** For Strategy A, resolve target prim and extract their `sourceId:doc:documentId`. For Strategy B source ids, use strings directly. Construct `ReferenceElement` entries.

USD-Native Concepts

These concepts exist in USD but have no equivalent in the AAS HandoverDocumentation metamodel and are dropped on USD → AAS export.

Composition Arcs

`references`, `payloads`, `inherits`, `specializes`, `over`. Flatten the composed prim before reading attribute values.

Time-Varying Attributes

Use `UsdAttribute::Get()` with `UsdTimeCode::Default()`. Fall back to `UsdTimeCode(0)`.

Variant Sets

Export the currently selected variant only.

Relationships (beyond AAS-mapped relationships)

`rel doc:documentedEntities`, `rel doc:refersToEntities`, `rel doc:basedOnReferences`, and `rel doc:translationOfEntities` have defined AAS equivalents and are exported per the reconstruction steps above. All other USD relationships on documentation prims are dropped.

Appendices

Appendix A: StatusValue Token Mapping

`doc:status` token values correspond 1:1 with AAS `StatusValue` string values — no lookup table required.

AAS StatusValue	USD Token	Description
Released	Released	Officially released and authoritative
InReview	InReview	Under review; not yet authoritative
Withdrawn	Withdrawn	Superseded or retracted

AAS → USD: write token value equal to the AAS string.
USD → AAS: write AAS string equal to the token value. If the token is not in this table, the USD → AAS export must fail validation or write a best-effort string with a warning.

Appendix B: Common VDI 2770:2020 Classification Codes

Use as values for `docClass:classId` when `docClass:classificationSystem = "VDI2770:2020"`.

ClassId	Description (EN)	Description (DE)
03-01	Assembly, disassembly, and commissioning instructions	Montage-, Aufbau- und Inbetriebnahmeanleitung
03-02	Operating manual	Betriebsanleitung
03-03	Safety instructions	Sicherheitsanleitung
03-04	Maintenance and servicing instructions	Wartungs- und Instandhaltungsanleitungen
03-05	Repair instructions	Reparaturanleitung
03-06	Spare parts catalogue	Ersatzteilmformationen
03-07	Training documentation	Schulungsunterlagen
04-01	Technical drawing	Technische Zeichnung
04-02	Wiring diagram / circuit diagram	Schaltplan
04-03	Bill of materials	Stückliste

ClassId	Description (EN)	Description (DE)
06-01	Certificates and declarations of conformity	Zertifikate und Konformitätserklärungen
07-01	Other	Sonstiges

Appendix C: Round-trip Fidelity Summary

AAS Field	Direction	Fidelity	Loss / Condition
Documents SML ordering	Both	Lossless	Requires <code>Document_NN</code> naming convention
Document AAS idShort	Both	Lossless	Preserved in <code>customData["aas:idShort"]</code>
DocumentDomainId / DocumentIdentifier	Both	Lossless	—
DocumentIsPrimary	Both	Lossless (when authored)	Optional; absent = not authored in USD
DocumentId SML ordering	Both	Lossless	Requires <code>DocumentId_NN</code> naming convention
Primary document ID (<code>sourceId:doc:documentId</code>)	Both	Lossless	Convenience projection; child prims authoritative
ClassificationSystem / ClassId	Both	Lossless	—
DocumentClassification SML ordering	Both	Lossless	Requires naming convention
ClassName (primary language)	Both	Lossless	—
ClassName (language variants)	Both	Lossy	Conditional on <code>customData</code> not stripped
DocumentedEntities (ExternalReference strings)	Both	Lossless	Via <code>sourceId:doc:documentedEntityIds</code>
DocumentedEntities (AAS ModelReference structure)	Both	Partial	Leaf identifier only
DocumentVersion SML ordering	Both	Lossless	Requires <code>DocumentVersion_NN</code> naming convention
Language (BCP 47 codes)	Both	Lossless	—
Version	Both	Lossless	—
Title / Description (primary language)	Both	Lossless	—
Title / Description (language variants)	Both	Lossy	Conditional on <code>customData</code> not stripped
Subtitle / KeyWords (primary language)	Both	Lossless	When authored
Subtitle / KeyWords (language variants)	Both	Lossy	Conditional on <code>customData</code> not stripped
StatusValue	Both	Lossless	1:1 token / string mapping; see Appendix A
StatusSetDate	Both	Lossless	Must be valid ISO 8601
OrganizationShortName / OrganizationOfficialName	Both	Lossless	—
DigitalFiles paths	Both	Lossless	—
DigitalFiles ordering	Both	Lossless	Array order preserved
DigitalFiles MIME types	Both	Lossy	Use <code>customData["doc:digitalFilesMimeTypes"]</code>

AAS Field	Direction	Fidelity	Loss / Condition
PreviewFile path	Both	Lossless	When authored
PreviewFile MIME type	Both	Lossy	Use <code>customData["doc:previewFileMimeType"]</code>
RefersToEntities / BasedOnReferences / TranslationOfEntities (intra-stage)	Both	Lossless	Requires prims + naming convention
RefersToEntities / BasedOnReferences / TranslationOfEntities (cross-stage)	Both	Lossless	Via <code>sourceId:doc:*</code> string attributes
ReferenceElement first/second structure	Both	Partial	Leaf identifier only
semanticIds	Both	Conditional	Survive only if <code>customData</code> not stripped
AAS Asset / AAS hierarchy	AAS → USD	Dropped	Not encoded in USD
Entities SML (VDI 2770 entity hierarchy)	AAS → USD	Dropped	USD prim hierarchy replaces this concept
Composition arcs	USD → AAS	Dropped	Flatten before export
Time-varying attributes	USD → AAS	Dropped	Use default value
Variant sets	USD → AAS	Dropped	Export selected variant

Appendix D: Cross-reference with DPP Battery Passport

The IDTA 02035-1 Digital Battery Passport Nameplate stores compliance document references as string identifier arrays:

- `sourceId:dpp:euDeclarationOfConformityIds` (string[]) — EU Declaration of Conformity document IDs
- `sourceId:dpp:testReportComplianceIds` (string[]) — test report document IDs

These values correspond to `docId:documentIdentifier` values in `DocumentIdAPI` entries within this submodel. When both submodels are co-located in the same USD stage, the link between battery passport identifier and handover documentation prim is explicit:

Linking rule: a battery passport's `sourceId:dpp:euDeclarationOfConformityIds` or `sourceId:dpp:testReportComplianceIds` value matches the `sourceId:doc:documentId` (and specifically a `docId:documentIdentifier` in the `DocumentIds/` hierarchy) of a `Document_NN` prim in the `Documents/` hierarchy.

Cross-referencing in USD → AAS direction: read the battery passport source id values, find matching `Document_NN` prims in `Documents/`, and note the cross-submodel relationship in the AAS envelope.